Parcial Algoritmos y Programación II - 7541 Curso Calvo (1º instancia) 19 de mayo de 2017

Padrón: **Apellido y Nombre:**

Punteros: APROBADO – DESAPROBADO **TDA:** APROBADO – DESAPROBADO

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main(){

  int *A, *C, *F;
  int **B;
  char *D, *E;
  char G;
  int H;

  H = 64 + ULTIMO_DIGITO_PADRON;
  G = 'A';
  A = &H;
  D = &G;
  B = &A;
  cout << *A <<  *D << **B << endl;

  C = new int[3];
  F = C + 2;
  C[1] = H;
  (*A) = 70;
  (*C) = H;
  (*F) = 68;
  C[2]++;
  cout << C[2] << C[1] << C[0] << endl;

  (**B) = C[0] + 1;
  A = C + 2;
  (**B) = H;
  C[0] = *A;
  (*F) = A[0] - 3;
  cout << C[2] << C[1] << C[0] << endl;

  E = (char*) A;
  C[1] = (**B) - 4;
  cout << (*D) << (*E) << G << endl;

  // liberar la memoria
  return 0;
}
```

2. Implementar para la clase Lista<T> con una estructura simplemente enlazada el siguiente método, indicando pre y post:

```
T remover(unsigned int posicionElemento);
```

3. Implementar el método `'crearLineaDeTiempoCombinandoHitos'` de la clase `'Historiador'` a partir de las siguientes especificaciones:

```
class Historiador {

  public:

    /* post: crea y devuelve una nueva LineaDeTiempo con el nombre indicado y con los Hitos que tienen las LineasDeTiempo
     *       'lineasDeTiempoPorCombinar' que ocurrieron en los años dados en 'aniosElegidos'.
     */
    LineaDeTiempo* crearLineaDeTiempoCombinandoHitos(string nombre,
                                                     Lista<LineaDeTiempo*>* lineasDeTiempoPorCombinar,
                                                     Lista<unsigned int>* aniosElegidos)
};
```

```
class LineaDeTiempo {

  public:

   /* post: linea de tiempo sin hitos asociados.
    */
    LineaDeTiempo(string nombre);

    string obtenerNombre();

    /* post: agrega nuevoHito a la linea de tiempo,
     *       ordenándolo temporalmente.
     */
    void incluir(Hito* nuevoHito);

    /* post: devuelve los Hitos ordenados temporalmente.
     */
    Lista<Hito*>* obtenerHitos();
};
```

```
class Hito {

  public:

    /* post: hito creado con la descripción y año indicados.
     */
    Hito(string descripcion, unsigned int anio);

    /* post: devuelve la descripción del hito.
     */
    string obtenerDescripcion();

    /* post: devuelve el año en el que sucedió el hito.
     */
    unsigned int obtenerAnio();
};
```

4. Diseñar la especificación e implementar el TDA **DispenserDeAgua**. Debe proveer operaciones para:

- Crear el dispenser con el caudal de agua [cm³ / segundo] que puede salir por su canilla.
- Colocar un bidón en el dispenser, especificando la cantidad de agua [litros] con la que está cargado.
- Retirar el bidón del dispenser.
- Devolver la cantidad de agua disponible [litros].
- Accionar la canilla, indicando el tiempo [segundos] y devolviendo la cantidad de agua entregada [cm³].
- Devolver el promedio de agua [cm³] entregada por vez.

Nota: Solo debe ser posible retirar el bidón del dispenser si está vacío. 1000 cm³ = 1 litro



Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas

Solución 1:

```
**0**
64A64
696470
686471
ADA
**1**
65A65
696570
686571
ADA
**2**
66A66
696670
686671
ADA
**3**
67A67
696770
686771
ADA
**4**
68A68
696870
686871
ADA
**5**
69A69
696970
686971
ADA
**6**
70A70
697070
687071
ADA
**7**
71A71
697170
687171
ADA
**8**
72A72
697270
687271
ADA
**9**
73A73
697370
687371
ADA
```

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas